

Aplicație Transport Public



Cuprins

1	Definirea cerintelor	2
2	Proiectarea bazei de date	2
2.1	ER Diagram	2
2.2	Reguli	3
3	Interfața Vizuală	3
3.1	Modul Hartă Navigație	3
3.2	Modul Orare	3
3.3	Modul Cont Bilete	3
3.4	Panou Administrator	3
4	Corelarea interfetei vizuale cu baza de date	6
4.1	Insert, update, delete	6
4.2	Interogari simple si complexe	7

1 Definirea cerintelor

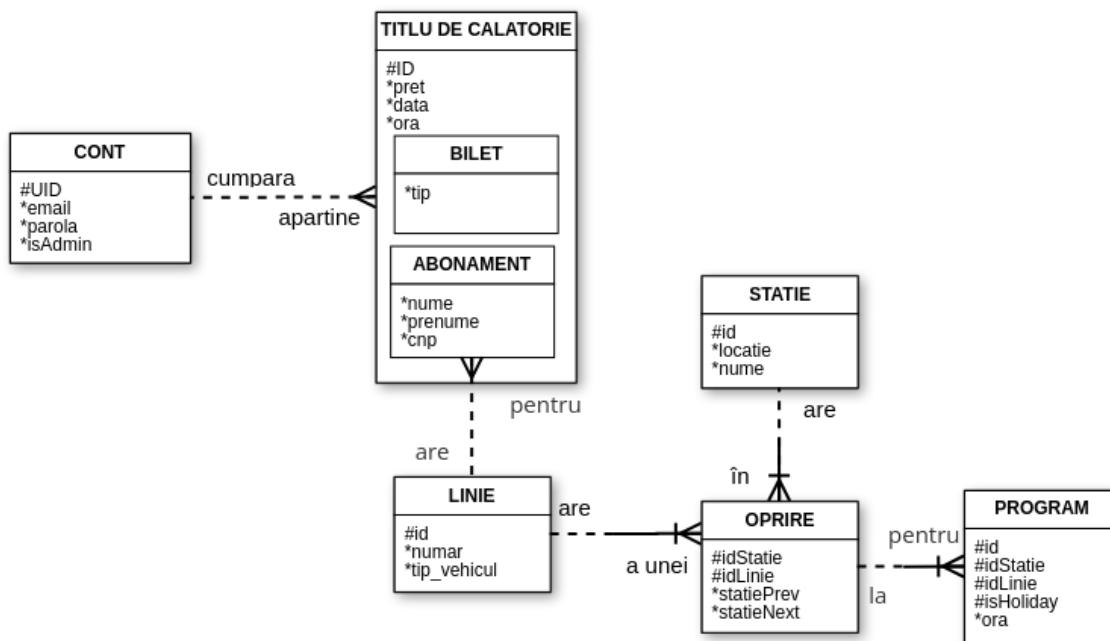
Utilizatorul va trebui să își creeze/ să se logheze într-un **cont** (pe care îl poate șterge/modifica) pentru a putea cumpăra **bilet/abonament**. Este necesară vizualizarea **hărții** orașului (integrată cu Open Street Map) cu **stațiile** și a listei cu toate **liniile** ce circulă în oraș, cât și a **programului** la care vehiculele sosesc în stație.

Stațiile împreună cu liniile formează un graf ponderat (costul fiind timpul parcurs între fiecare stație de pe linie, dedus din program) ce ne ajută în implementarea unui algoritm de **navigare**, cu ajutorul căruia utilizatorul poate calcula o rută dintr-o locație în alta.

Altă funcționalitate este, cu ajutorul unui cont de **administrator**, posibilitatea editării liniilor, stațiilor și a programelor.

2 Proiectarea bazei de date

2.1 ER Diagram



2.2 Reguli

Reguli implementate în logica bazei de date:

- UNQ: email, cnp, nume statie, numar linie, toate IDurile
- isAdmin → cont de Administrator
- Abonamentul este activ timp de 30 de zile de la data reincarcarii
- 4 tipuri de bilete: 3h, 24h, 72h, 7d

3 Interfața Vizuală

3.1 Modul Hartă Navigație

- Hartă interactivă centrată pe Constanța
- Selecție linii prin meniul burger ()
- Planificare rută cu introducere stații de plecare/destinație
- Vizualizare stații pe hartă cu marker-uri colorate
- Trasare rute cu linii roșii și instrucțiuni detaliate
- Toggle vizibilitate stații și linii

3.2 Modul Orare

- Listare toate liniile în combobox
- Vizualizare orar pentru fiecare linie selectată
- Afisare timpi sosire pentru fiecare stație
- Calcul timp până la următoarea sosire (colorat: verde < 5min, portocaliu < 15min, roșu > 15min)

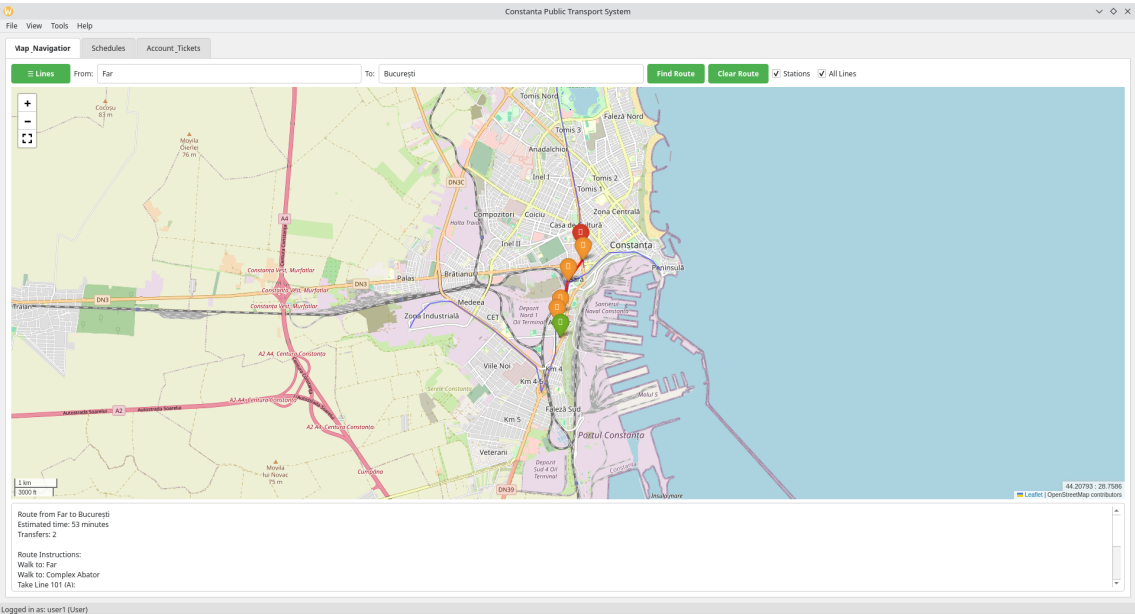
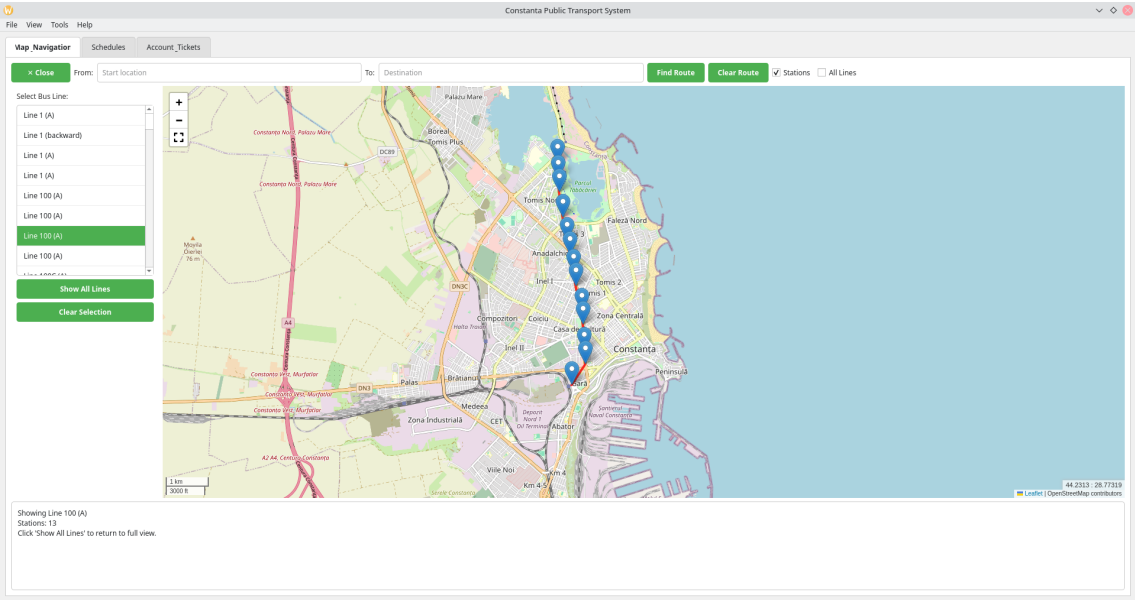
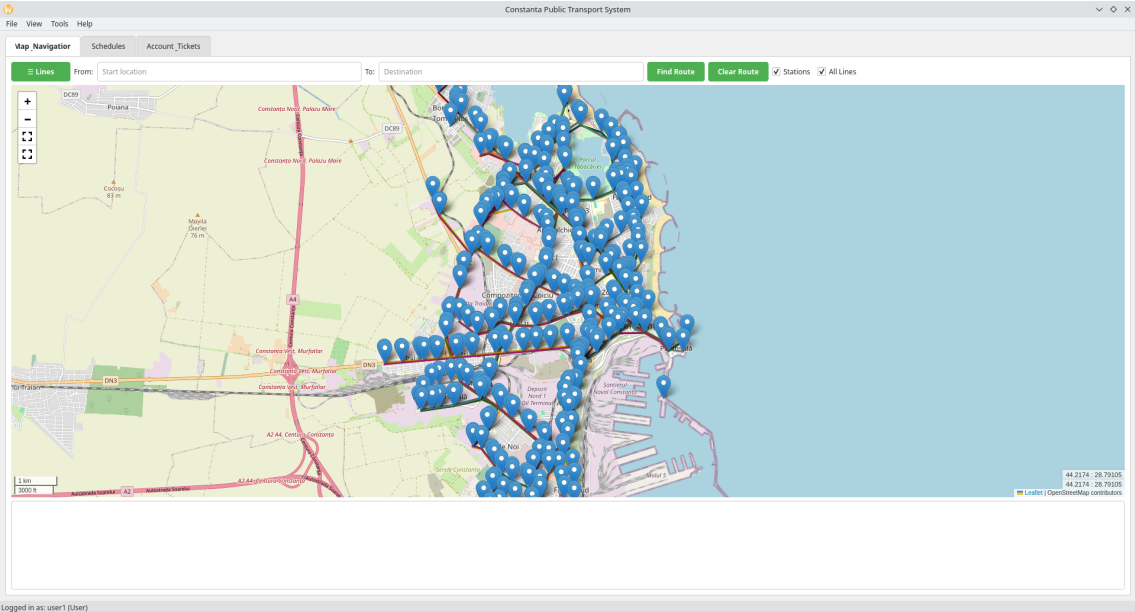
3.3 Modul Cont Bilete

- Înregistrare cont nou (username, email, parolă)
- Autentificare utilizator existent
- Istoric călătorii cu detalii despre fiecare călătorie
- Cumpărare bilet (tipuri: 3h, 24h, 72h, 7d)
- Cumpărare abonament (general sau pentru linie specifică)
- Editare cont (schimbare username, email, parolă)
- Ștergere cont cu confirmare

3.4 Panou Administrator

- Gestionare stații (adaugare, editare, ștergere)
- Gestionare linii (adaugare, editare)
- Gestionare orare (editare timpi pentru fiecare stație-linie)
- Vizualizare abonamente
- Acces restricționat doar pentru utilizatori cu rol 'admin'

Aplicație Transport Public

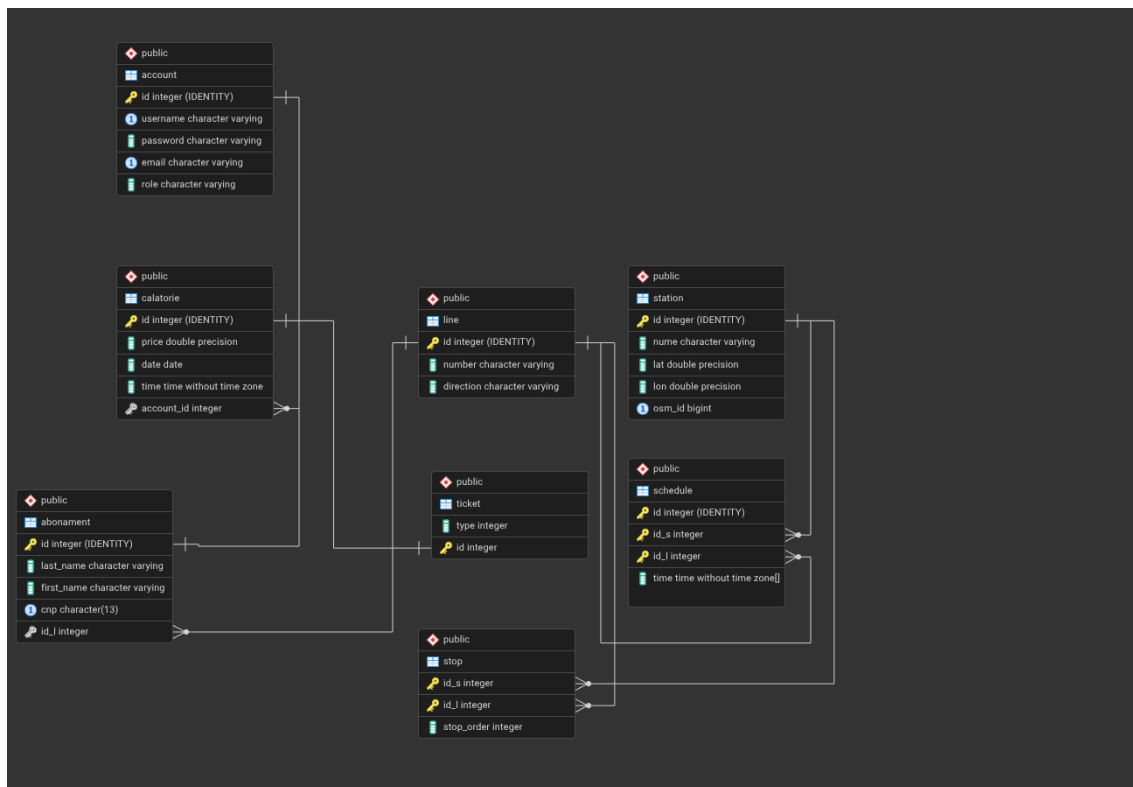


Aplicație Transport Public

Public Transport System			
File	View	Line 101 (A)	
Map_Nav		Line 101M (A)	
		Line 101M (A)	
Map_Nav		Line 101M (A)	
Select Line		Line 101M (A)	
		Line 102 (A)	
		Line 102 (A)	
1	Indur	Line 102 (A)	08:00 (in 4 min)...
2	Celco	Line 13 (A)	08:00 (in 4 min)...
3	CT B	Line 13 (A)	08:00 (in 4 min)...
4	Parc	Line 13 (A)	08:00 (in 4 min)...
5	Cum	Line 13 (A)	08:00 (in 4 min)...
6	Spiru	Line 14 (A)	08:00 (in 4 min)...
7	I.C. B	Line 14 (A)	08:00 (in 4 min)...
8	Dezr	Line 14 (A)	08:00 (in 4 min)...
9	Dum	Line 2-43 (A)	08:00 (in 4 min)...
10	I.L. C	Line 2-43 (A)	08:00 (in 4 min)...
11	Popo	Line 2-43 (A)	08:00 (in 4 min)...
12	Casa	Line 3 (A)	08:00 (in 4 min)...
13	Colg	Line 3 (A)	08:00 (in 4 min)...
14	Enel	Line 3 (A)	08:00 (in 4 min)...
15	Scoai	Line 3 (A)	08:00 (in 4 min)...
16	Univ	Line 3B (A)	08:00 (in 4 min)...
17	Ion R	Line 3B (A)	08:00 (in 4 min)...
18	Fäger	Line 3B (A)	08:00 (in 4 min)...
19	Scoai	Line 42 (A)	08:00 (in 4 min)...
20	Tabla	Line 42 (A)	08:00 (in 4 min)...
21	Cajin	Line 42 (A)	08:00 (in 4 min)...
22	Direc	Line 43C (A)	08:00 (in 4 min)...
23	Pesc	Line 43C (A)	08:00 (in 4 min)...
24	Gard	Line 43C (A)	08:00 (in 4 min)...
25	Pesc	Line 43C (A)	08:00 (in 4 min)...
26	Sat V	Line 43M (A)	08:00 (in 4 min)...
27	Cap L	Line 43M (A)	08:00 (in 4 min)...
28	Cap L	Line 43M (A)	08:00 (in 4 min)...

The screenshot displays the Constanta Public Transport System web application. The top navigation bar includes links for File, View, Tools, and Help. Below this, a secondary navigation bar features buttons for Map_Navigation, Schedules, and Account_Ticket, with the latter being the active page. The main content area is titled "My Account" and contains a large, empty white space. In the center of this space, the text "Logged in as: user1" and "Role: User" is displayed. At the bottom of the page, a vertical sidebar contains a list of account-related actions: Travel History, Buy Ticket, Buy Subscription, Edit Account, Delete Account (highlighted in red), and Logout. The bottom status bar indicates the user is logged in as "user1 (User)".

4 Corelarea interfetei vizuale cu baza de date



Pentru a pregăti baza de date pentru funcționarea aplicației, am inițializat structura de date prin intermediul API-ului OpenStreetMap, extragând și încărcând automat toate stațiile de autobuz existente în Constanța, împreună cu liniile de transport corespunzătoare și ordinea opririlor pentru fiecare rută. Ulterior, pentru a completa sistemul cu informații operationale esențiale, am rulat un script care a calculat și inserat programele de sosire pentru fiecare stație pe fiecare linie, asigurând astfel că aplicația poate oferi utilizatorilor informații precise despre orele de plecare și sosire ale autobuzelor. Această abordare în două etape a permis crearea unei baze de date complete și realiste fără a necesita introducere manuală extinsă a datelor.

Frontend: PyQt5 pentru interfața grafică

Mapping: OpenStreetMap + Folium pentru vizualizare hărți

Backend: Python 3.x

Bază de Date: PostgreSQL cu psycopg2

Algoritmi: Dijkstra pentru găsirea rutelor optime

4.1 Insert, update, delete

Insert:

1. Inserare cont utilizator nou

```
INSERT INTO account (username, password, email, role)
VALUES ('nume_utilizator', 'parolă', 'email@exemplu.ro', 'user')
RETURNING id;
```

2. Inserare stație nouă

```
INSERT INTO station (nume, lat, lon, osm_id)
VALUES ('Nume Stație', 44.123456, 28.654321, 123456789)
RETURNING id;
```

3. Inserare bilet

```
INSERT INTO ticket (type) VALUES (1) RETURNING id;
```

4. Inserare călătorie (tranzacție)

```
INSERT INTO calatorie (account_id, price, date, time)
VALUES ([user_id], 5.00, CURRENT_DATE, CURRENT_TIME)
RETURNING id;
```

5. Inserare abonament (cu sau fără linie specifică)

```
INSERT INTO abonament (last_name, first_name, cnp, id_l)
VALUES ('Popescu', 'Ion', '1234567890123', [line_id_or_null])
RETURNING id;
```

6. Inserare abonament cu ID explicit (fallback)

```
INSERT INTO abonament (id, last_name, first_name, cnp, id_l)
VALUES ([next_id], 'Popescu', 'Ion', '1234567890123', [line_id_or_null]);
```

Update:

1. Actualizare informații cont utilizator

```
UPDATE account
SET username = 'nume_nou', email = 'email_nou@exemplu.ro', password = 'parolă_nouă'
WHERE id = [user_id];
```

2. Actualizare abonament (schimbare linie)

```
UPDATE abonament
SET id_l = [new_line_id]
WHERE id = [subscription_id];
```

3. Actualizare stare bilet (utilizat)

```
UPDATE ticket
SET is_used = TRUE
WHERE id = [ticket_id];
```

Delete:

1. Ștergere cont utilizator

```
DELETE FROM account WHERE id = [user_id];
```

2. Ștergere abonamente expirate

```
DELETE FROM abonament WHERE valid_until < CURRENT_DATE;
```

4.2 Interogari simple si complexe

Interogari Simple:

1. Selectare stații pentru o linie specifică cu ordinea opririlor

```
SELECT s.*, st.stop_order
FROM stop st
JOIN station s ON st.id_s = s.id
WHERE st.id_l = [line_id]
ORDER BY st.stop_order;
```

2. Selectare orar pentru o anumită stație și linie

```
SELECT time FROM schedule
WHERE id_l = [line_id] AND id_s = [station_id];
```

3. Selectare călătorii pentru un utilizator specific

```
SELECT * FROM calatorie
WHERE account_id = [user_id]
ORDER BY date DESC, time DESC;
```

4. Selectare abonamente cu informații linie

```
SELECT a.*, l.number as line_number
```

```
FROM abonament a
LEFT JOIN line l ON a.id_l = l.id
ORDER BY a.last_name, a.first_name;
```

5. Selectare stație după nume (căutare parțială)

```
SELECT * FROM station
WHERE LOWER(nume) LIKE LOWER('%nume_stație%')
LIMIT 1;
```

6. Selectare ultimul ID din abonamente

```
SELECT COALESCE(MAX(id), 0) FROM abonament;
```

Interogari Complexe:

1. Selectare toate liniile cu numărul de stații și stația de plecare

```
SELECT
    l.id,
    l.number,
    l.direction,
    COUNT(DISTINCT st.id_s) as station_count,
    MIN(s1.nume) as first_station,
    MAX(s2.nume) as last_station
FROM line l
LEFT JOIN stop st ON l.id = st.id_l
LEFT JOIN station s1 ON st.id_s = s1.id AND st.stop_order = (
    SELECT MIN(st2.stop_order)
    FROM stop st2
    WHERE st2.id_l = l.id
)
LEFT JOIN station s2 ON st.id_s = s2.id AND st.stop_order = (
    SELECT MAX(st3.stop_order)
    FROM stop st3
    WHERE st3.id_l = l.id
)
GROUP BY l.id, l.number, l.direction
ORDER BY l.number;
```

2. Selectare stații cu toate liniile care le deservesc

```
SELECT
    s.id,
    s.nume,
    ARRAY_AGG(DISTINCT l.number ORDER BY l.number) as lines_serving,
    COUNT(DISTINCT l.id) as line_count
FROM station s
JOIN stop st ON s.id = st.id_s
JOIN line l ON st.id_l = l.id
GROUP BY s.id, s.nume
HAVING COUNT(DISTINCT l.id) > 1 Stații servite de mai multe linii
ORDER BY line_count DESC;
```

3. Calcul timp mediu între stații pentru fiecare linie

```
SELECT
    l.number,
    l.direction,
    ROUND(AVG(
        EXTRACT(EPOCH FROM (sc2.time[1]::time - sc1.time[1]::time)) / 60
    ), 1) as avg_time_minutes,
    COUNT(*) as segment_count
FROM line l
JOIN stop st1 ON l.id = st1.id_l
JOIN stop st2 ON l.id = st2.id_l AND st2.stop_order = st1.stop_order + 1
```



```
JOIN schedule sc1 ON l.id = sc1.id_l AND st1.id_s = sc1.id_s
JOIN schedule sc2 ON l.id = sc2.id_l AND st2.id_s = sc2.id_s
WHERE sc1.time[1] IS NOT NULL AND sc2.time[1] IS NOT NULL
GROUP BY l.id, l.number, l.direction
ORDER BY l.number;
```

4. Statistici utilizatori și tranzacții

```
SELECT
    a.username,
    a.email,
    a.role,
    COUNT(c.id) as travel_count,
    COALESCE(SUM(c.price), 0) as total_spent,
    MAX(c.date) as last_travel_date
FROM account a
LEFT JOIN calatorie c ON a.id = c.account_id
GROUP BY a.id, a.username, a.email, a.role
ORDER BY total_spent DESC;
```

5. Găsire stații comune între linii (puncte de transfer)

```
SELECT
    s.num as station_name,
    ARRAY_AGG(DISTINCT l1.number ORDER BY l1.number) as lines_connected,
    COUNT(DISTINCT l1.id) as connection_count
FROM station s
JOIN stop st1 ON s.id = st1.id_s
JOIN line l1 ON st1.id_l = l1.id
WHERE EXISTS (
    SELECT 1
    FROM stop st2
    JOIN line l2 ON st2.id_l = l2.id
    WHERE st2.id_s = s.id
    AND l2.id != l1.id
)
GROUP BY s.id, s.num
HAVING COUNT(DISTINCT l1.id) >= 2
ORDER BY connection_count DESC, s.num;
```

6. Analiza abonamente pe linii

```
SELECT
    l.number as line_number,
    l.direction,
    COUNT(a.id) as subscription_count,
    ROUND(COUNT(a.id) * 20.00, 2) as estimated_revenue,
    ARRAY_AGG(DISTINCT a.last_name || ' ' || a.first_name) as subscribers
FROM line l
LEFT JOIN abonament a ON l.id = a.id_l
GROUP BY l.id, l.number, l.direction
ORDER BY subscription_count DESC;
```

7. Detectie stații fără orare

```
SELECT
    s.id,
    s.num,
    ARRAY_AGG(DISTINCT l.number) as lines_without_schedule
FROM station s
JOIN stop st ON s.id = st.id_s
JOIN line l ON st.id_l = l.id
WHERE NOT EXISTS (
    SELECT 1
    FROM schedule sc
```

```
        WHERE sc.id_l = l.id AND sc.id_s = s.id
    )
    GROUP BY s.id, s.numé
    ORDER BY s.numé;
```

8. Calcul distanțe între stații consecutive (folosind formula haversine)

```
WITH station_pairs AS (
    SELECT
        l.id as line_id,
        l.number,
        s1.id as station1_id,
        s1.numé as station1_name,
        s1.lat as lat1,
        s1.lon as lon1,
        s2.id as station2_id,
        s2.numé as station2_name,
        s2.lat as lat2,
        s2.lon as lon2,
        st1.stop_order
    FROM line l
    JOIN stop st1 ON l.id = st1.id_l
    JOIN stop st2 ON l.id = st2.id_l AND st2.stop_order = st1.stop_order + 1
    JOIN station s1 ON st1.id_s = s1.id
    JOIN station s2 ON st2.id_s = s2.id
)
SELECT
    number,
    station1_name,
    station2_name,
    stop_order,
    ROUND(
        6371 * ACOS(
            COS(RADIANS(lat1)) * COS(RADIANS(lat2)) *
            COS(RADIANS(lon2) - RADIANS(lon1)) +
            SIN(RADIANS(lat1)) * SIN(RADIANS(lat2))
        ) * 1000
    ) as distance_meters
FROM station_pairs
ORDER BY number, stop_order;
```

9. Construire graf de transport pentru algoritmul Dijkstra

```
WITH graph_edges AS (
    SELECT
        s1.id as from_station_id,
        s2.id as to_station_id,
        l.id as line_id,
        l.number as line_number,
        l.direction,
        ROUND(
            AVG(
                EXTRACT(EPOCH FROM (sc2.time[1]::time - sc1.time[1]::time)) / 60
            )
        ) as travel_time_minutes
    FROM line l
    JOIN stop st1 ON l.id = st1.id_l
    JOIN stop st2 ON l.id = st2.id_l AND st2.stop_order = st1.stop_order + 1
    JOIN station s1 ON st1.id_s = s1.id
    JOIN station s2 ON st2.id_s = s2.id
    JOIN schedule sc1 ON l.id = sc1.id_l AND s1.id = sc1.id_s
    JOIN schedule sc2 ON l.id = sc2.id_l AND s2.id = sc2.id_s
    WHERE sc1.time[1] IS NOT NULL AND sc2.time[1] IS NOT NULL
```

```
GROUP BY s1.id, s2.id, l.id, l.number, l.direction
)
SELECT * FROM graph_edges
ORDER BY line_number, from_station_id;
```

10. Găsire stații în proximitate (pentru transferuri)

```
SELECT
    s1.id as station1_id,
    s1.nume as station1_name,
    s2.id as station2_id,
    s2.nume as station2_name,
    ROUND(
        6371 * ACOS(
            COS(RADIANS(s1.lat)) * COS(RADIANS(s2.lat)) *
            COS(RADIANS(s2.lon) - RADIANS(s1.lon)) +
            SIN(RADIANS(s1.lat)) * SIN(RADIANS(s2.lat))
        ) * 1000
    ) as distance_meters,
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM stop st1
            JOIN stop st2 ON st1.id_l = st2.id_l
            WHERE st1.id_s = s1.id AND st2.id_s = s2.id
        ) THEN 'same_line'
        WHEN EXISTS (
            SELECT 1 FROM stop st1 WHERE st1.id_s = s1.id
            INTERSECT
            SELECT 1 FROM stop st2 WHERE st2.id_s = s2.id
        ) THEN 'common_line'
        ELSE 'different_lines'
    END as connection_type
FROM station s1
CROSS JOIN station s2
WHERE s1.id < s2.id
AND 6371 * ACOS(
    COS(RADIANS(s1.lat)) * COS(RADIANS(s2.lat)) *
    COS(RADIANS(s2.lon) - RADIANS(s1.lon)) +
    SIN(RADIANS(s1.lat)) * SIN(RADIANS(s2.lat))
) <= 0.5 - 500 metri rază
ORDER BY distance_meters;
```